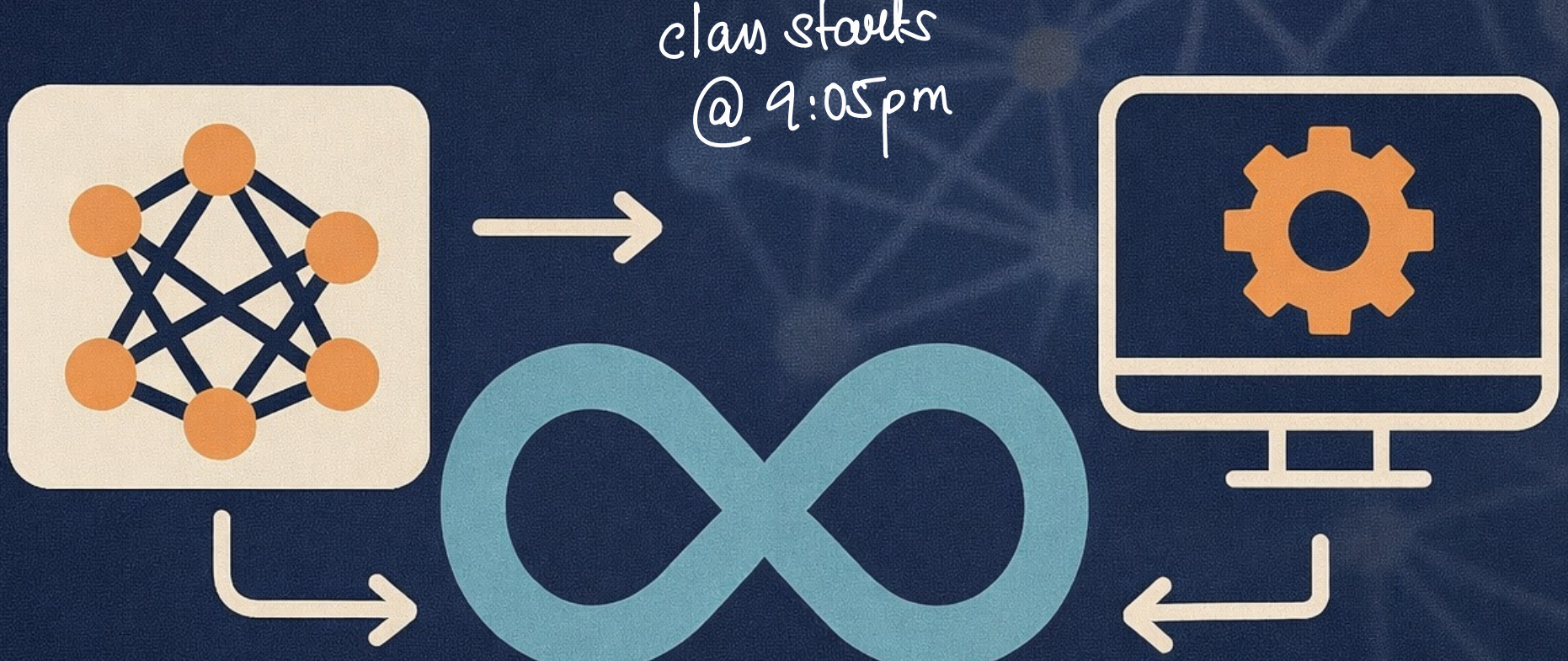
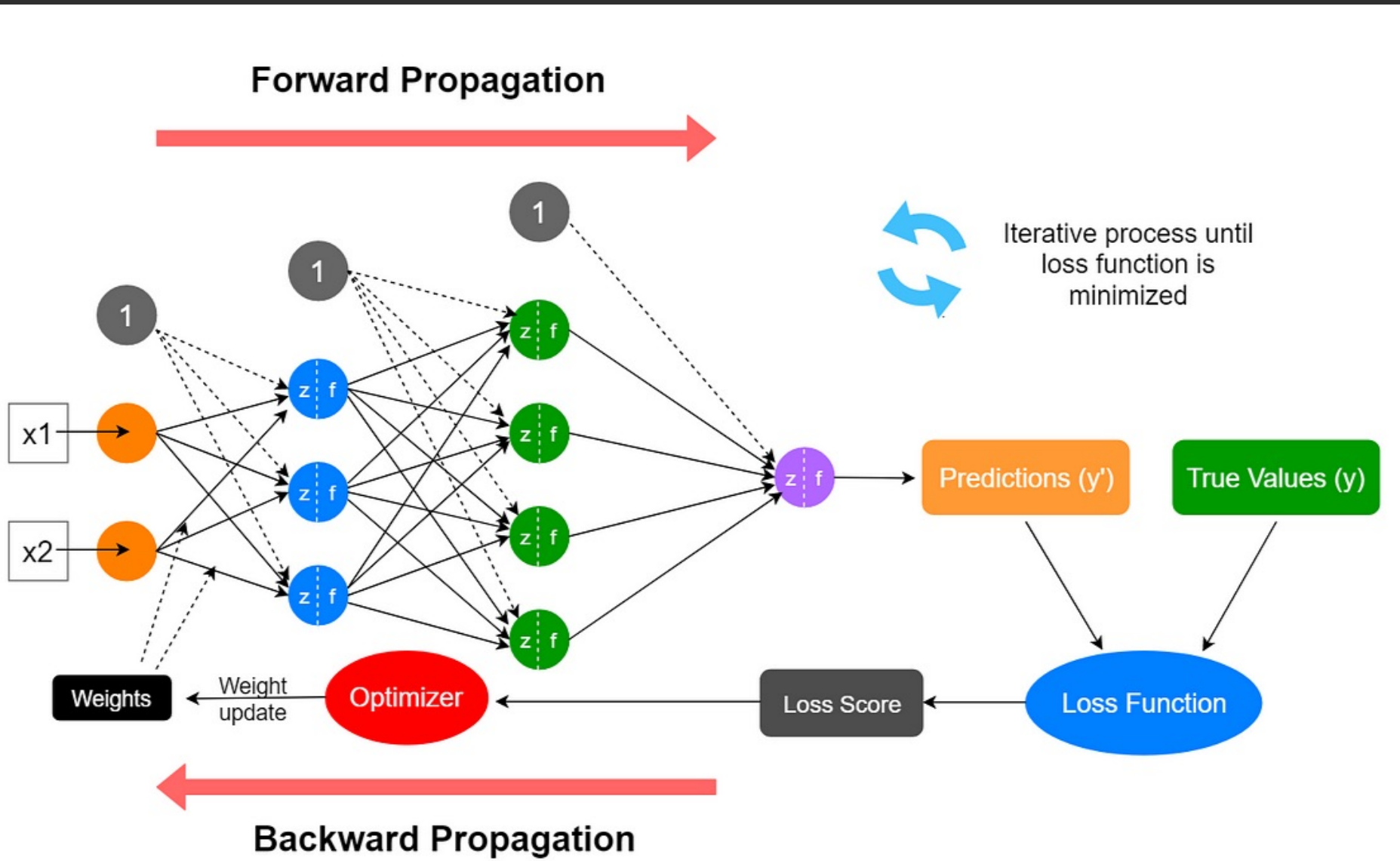
NEURAL NETWORK



Lecture : Neural Network Optimization

#Agenda

1) Forward & Backward prop code (numpy)
2) Weight Initialization
3) Optimizer & Activation function


Forward Propagation
1
1
1
x1
x2
z f
z f
z f
z f
z f
z f
z f
Iterative process until loss function is minimized
Predictions (y')
True Values (y)
Loss Function
Loss Score
Optimizer
Weight update
Weights
Backward Propagation

Forward pass

$z = w^T x + b$

$actfun(z)$

Activation Fun^n

softmax

$$\frac{e^z}{\sum e^{z_i}}$$

Bring the total prob to 1

$X \Rightarrow 3 \times 2$

$W \Rightarrow 2 \times 3$

$$w^T x \longrightarrow (3 \times 2) \cdot (2 \times 3) = 3 \times 3$$

Multiclass problem Classification

A	B	C
33%	33%	34%

softmax

Binary classificatio

A	B
0.6	0.4

sigmoid

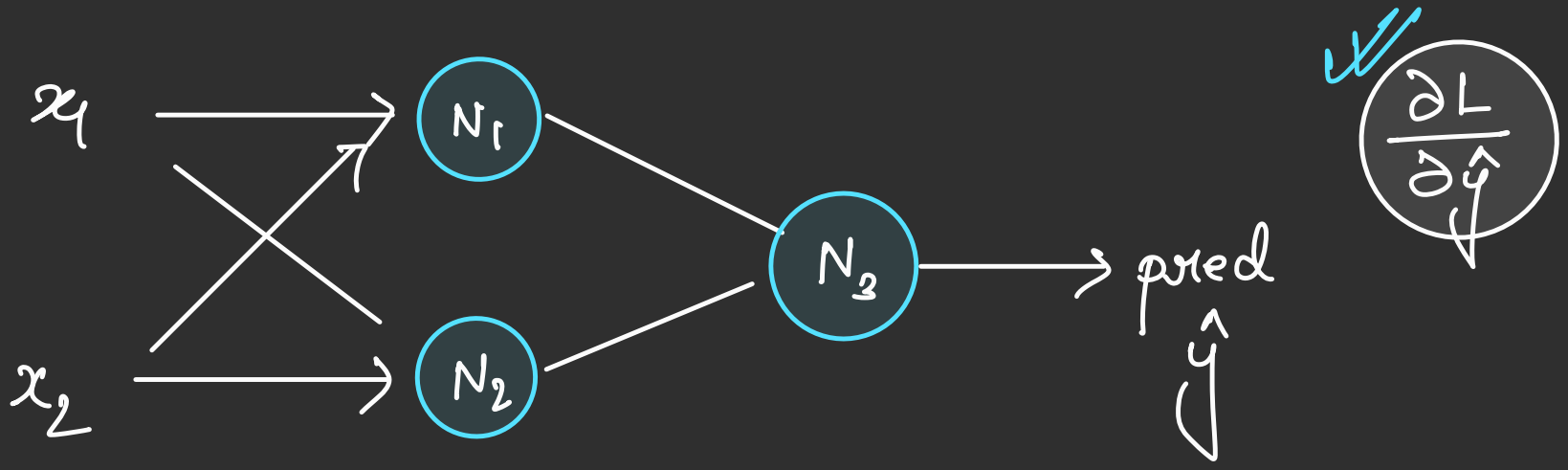
Log loss / Binary Cross Entropy

Binary class

Categorical Cross Entropy

Multi class

L → Loss function

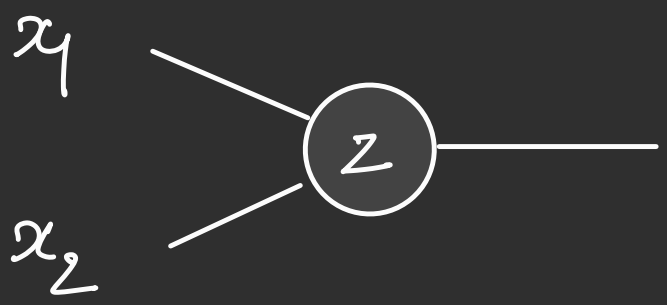


"GD needs it"

$\frac{\partial L}{\partial \hat{y}} =$

Loss function — Cross Entropy

$\frac{\partial L}{\partial \hat{y}} \Rightarrow$ pred − truth



$z = w_1 x_1 + w_2 x_2 + b$

$z = \text{activation_fun}^n(z)$

↳ output = $\hat{y}$ (prediction)

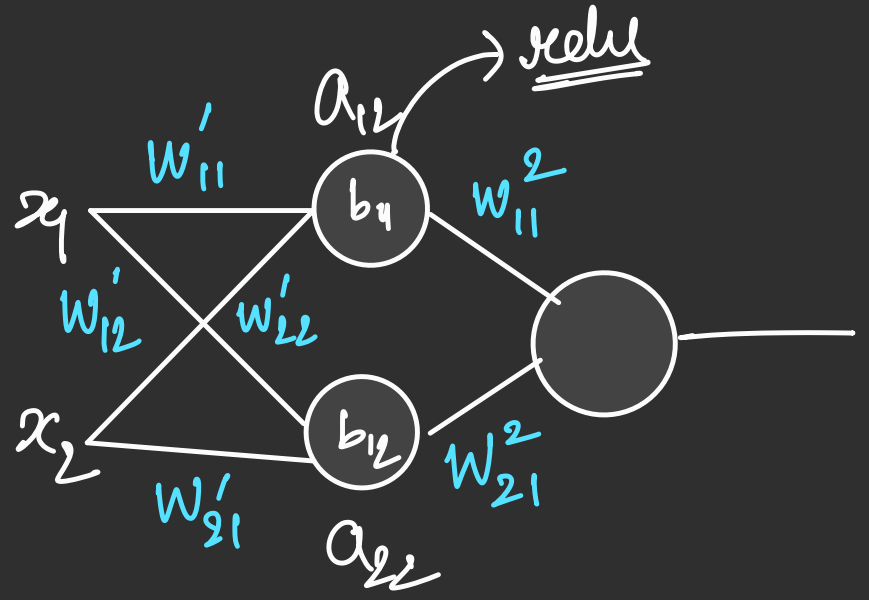
Weight Initialization

$$relu = \max(0, z)$$

Case I - Zero Initialization

$$z_{11} = w'_{11} x_1 + w'_{22} x_2 + b_1$$

$$\Rightarrow 0$$



$$\begin{cases} a_{12} = relu(z_{11}) = 0 \\ a_{22} = 0 \end{cases}$$

no update

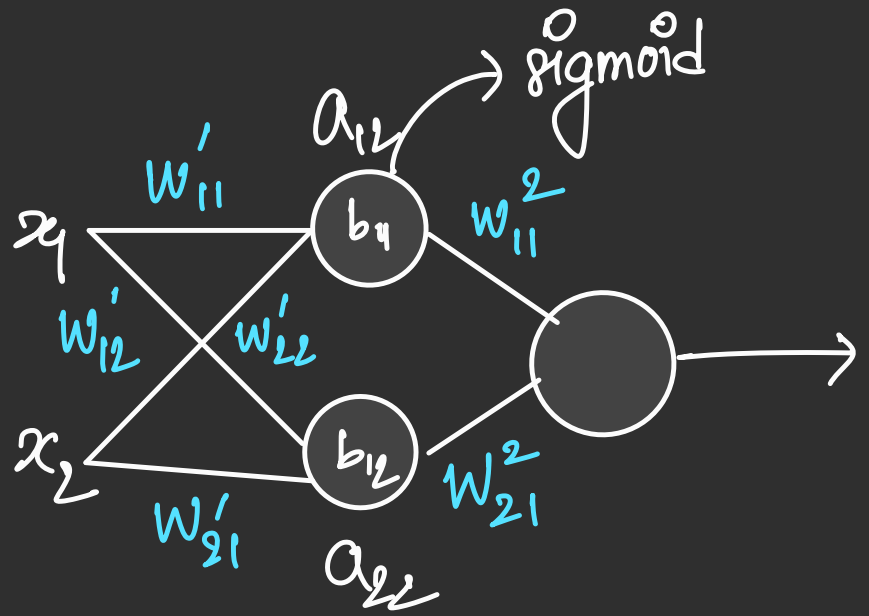
$(w, b = 0)$

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w_{old}}$$

"No training"

Case 2 : Same Value for all weights & bias

$w = 1 \quad b = 1$



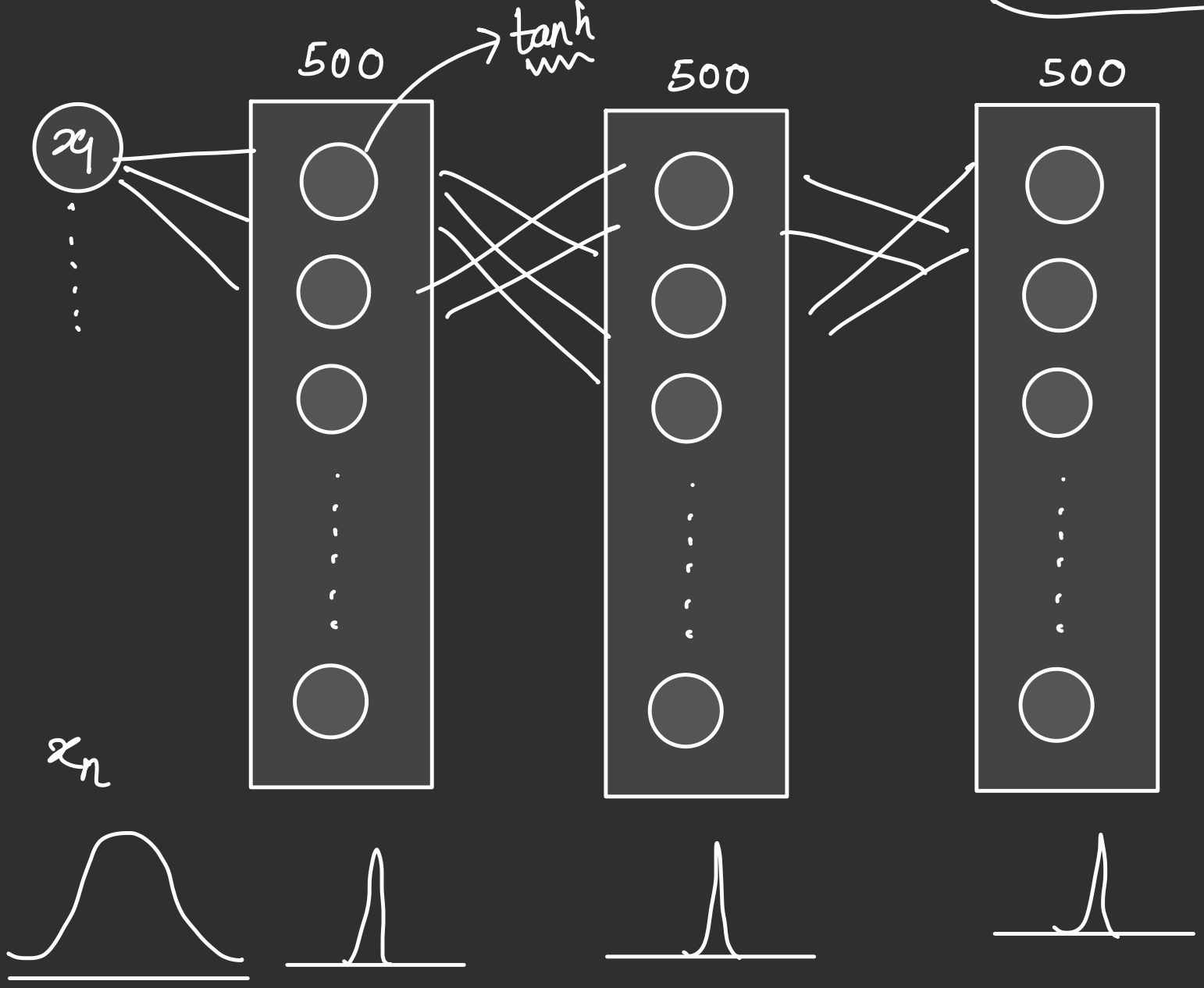
$$z_{11} = w_1 x_1 + w_2 x_2 + b$$

$$\begin{cases} a_{11} = 0.5 \\ a_{12} = 0.5 \end{cases}$$

Case 3: Randomness

np.random.randn(500, 500) x 0.01

Random but small values



"Vanishing gradient problem"

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

$$w_{new} = w_{old} - (\text{something})$$

$$\frac{\partial L}{\partial w_{old}}$$

small ⟶ smaller

$$3.02 \Rightarrow 3.02 - 0.0001$$

$W = 0.01$

$X = 2$

$W^T x = 2 \times 0.01 \Rightarrow 0.02$

$3 \times 0.01 \Rightarrow$

Solution

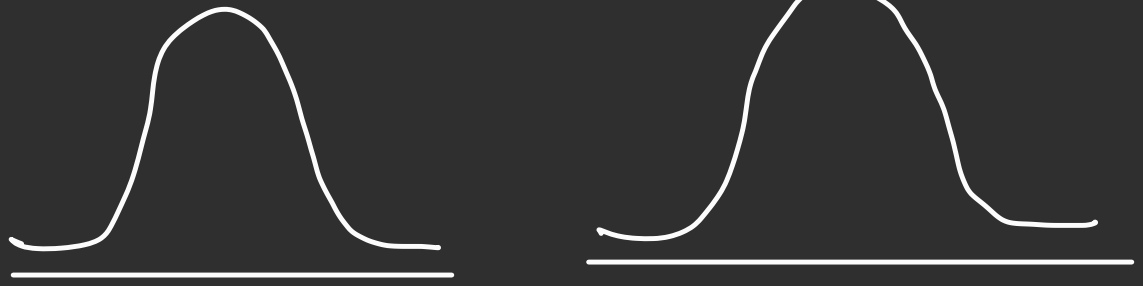
1) Glorot Initialization

n_{in} = number of Input neurons.

$$W \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right)$$

μ → 0

variance → $\frac{1}{n_{in}}$



Doubts

1) Activation Fun^{n} list

Sigmoid, tanh, relu, leaky relu, ELU

2) Neural Network

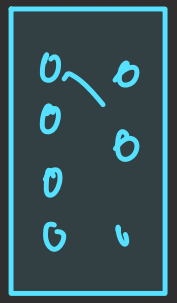
pattern

- CNN (Images)
- RNN (Text)
  - LSTM
    - Transformer

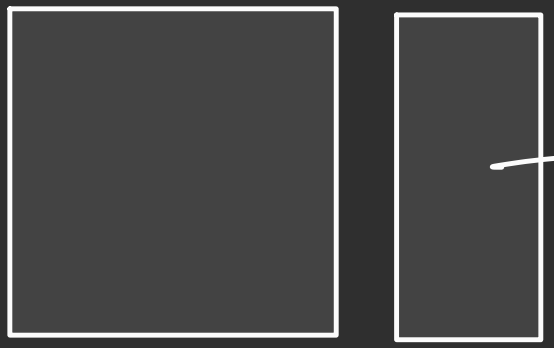
No — pytorch (ML framework)

LLMs → 1.2B parameters

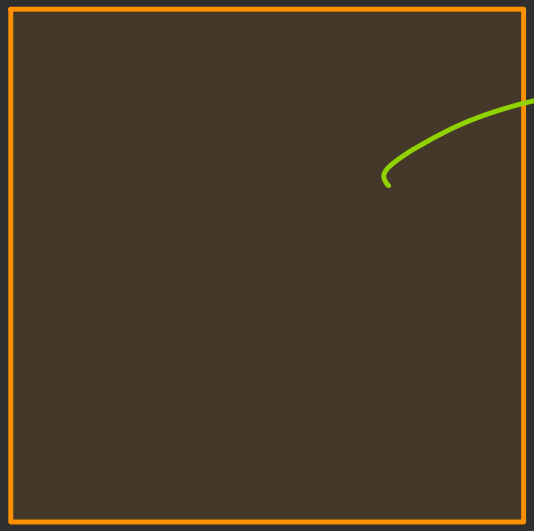
gradient = 500



Foundational brick



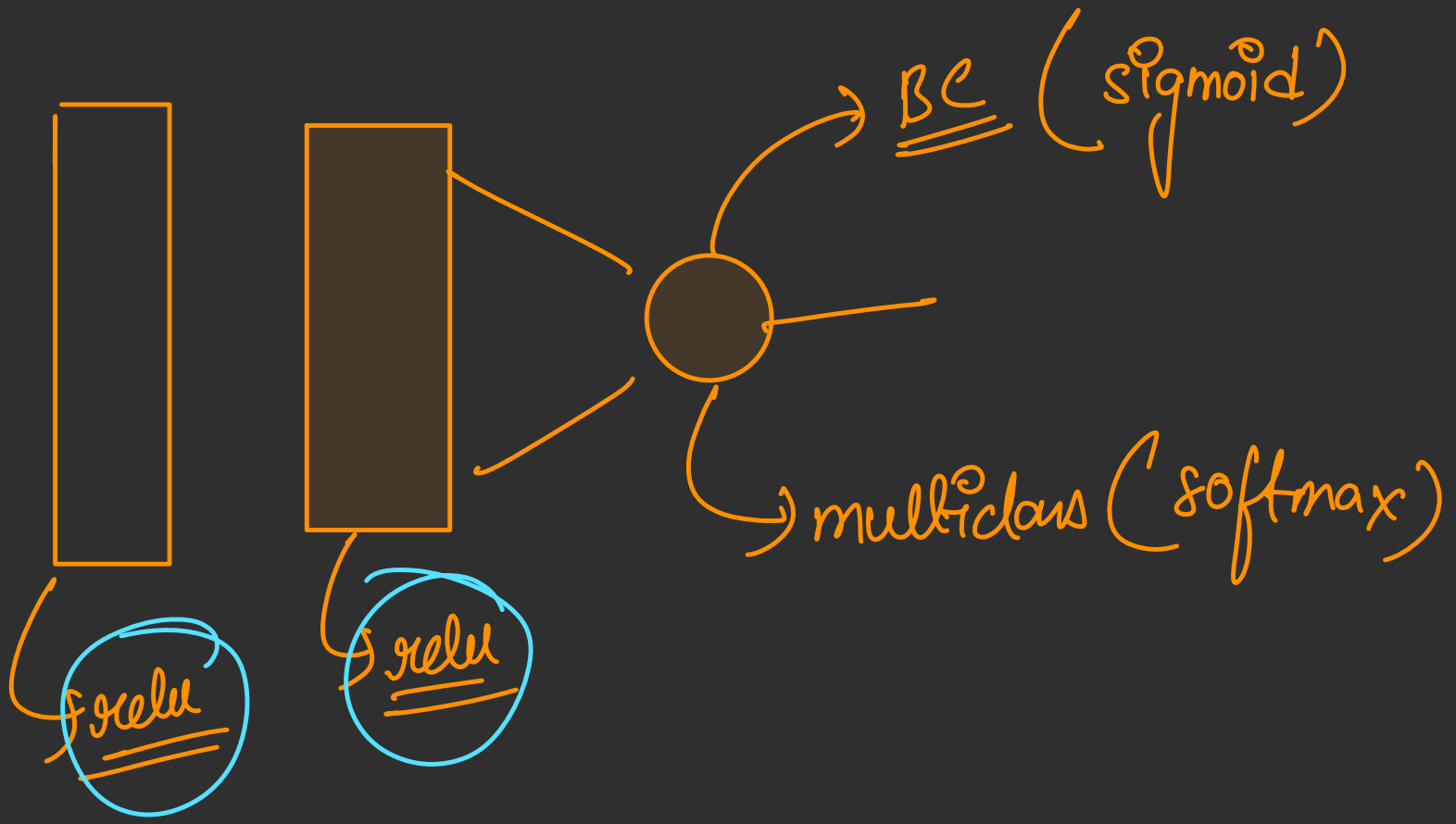
Our Embedd



Brick (Fur ther)

(Transfer Learning)

1) How to choose activation fun^{n}



tanh sigmoid → vanishing

Why to rescale & shift



$(\mu=0, \sigma=1)$

↳ Stabilize training

γ & β

$y = \gamma x + \beta$

γ ↳ Trainable parameter

↳ Allow the network to learn the best distribution again

Agentic Platform

Solution Architech

MLE

NN & Clasical ML

Trying & Test multiple architetture

Xgboost / RF / Isolation Forest

Time Series

TF / IFD

Signal Times Series

Agent

UI (Teams)

- Langgraph
- Langchain
- knowledge graph

Meetings + Code Reiew

"Data pre-processing" → 40%

"Design"